

Introduction to Robotics

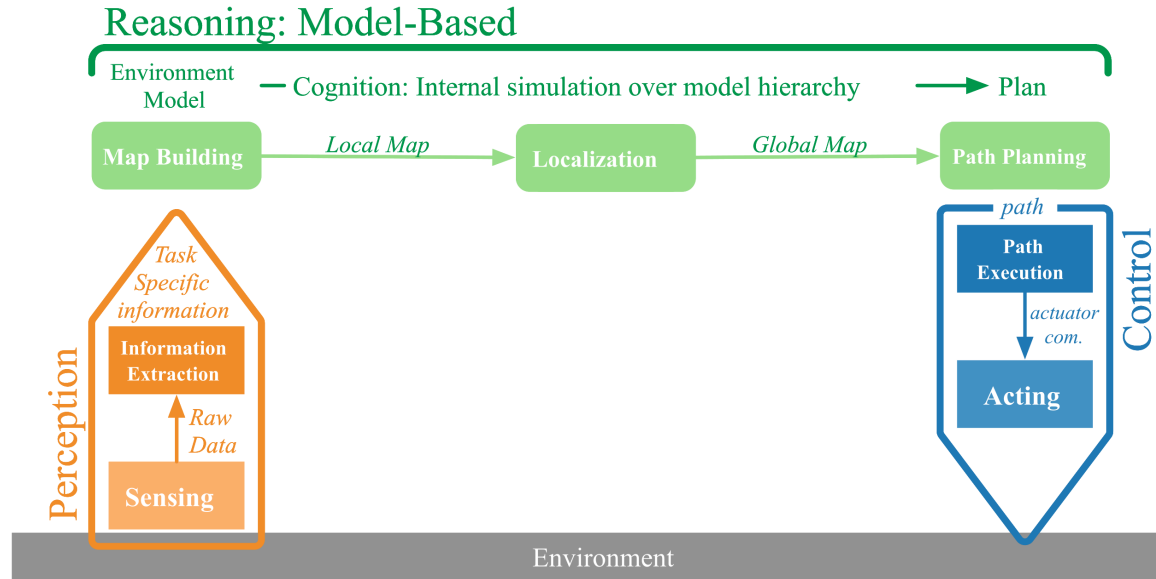


Universität
Münster

The PRA Loop

Actuation as the closing step of the control loop

Perceive · Reason · Act



- *Perceive*: read and process sensor data from the world
- *Reason*: decide what to do (planning, control, learning)
- *Act*: send commands to and move actuators

ROS2 Communication Patterns

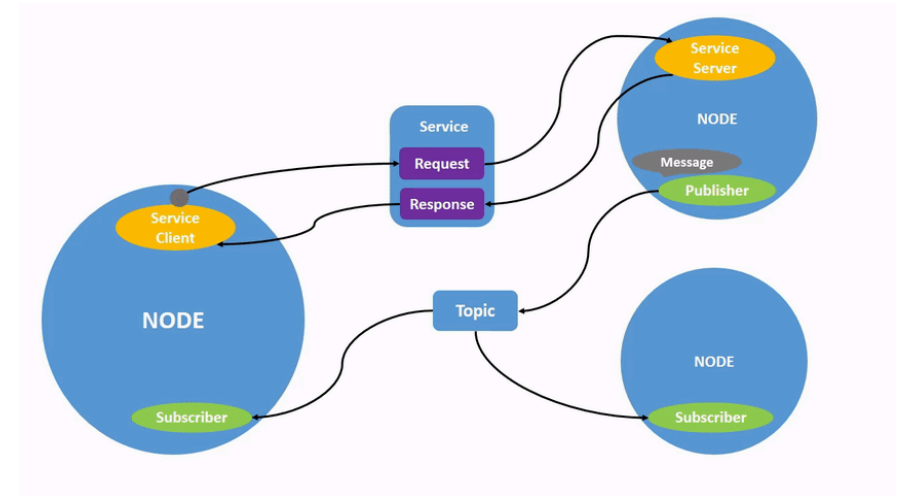
How nodes exchange information

The ROS2 Graph

ROS2 is a network of nodes

- A **node** is a single process with one responsibility (sensing, control, planning, ...)
- Nodes communicate through named channels - no direct calls
- The set of running nodes and connections is called the **ROS2 graph**

Four communication patterns: **topics** · **services** · **actions** · **parameters**



Interfaces

Typed contracts between nodes

- Every channel is **strongly typed** - sender and receiver share the same data definition
- Defined in plain-text **interface files**, compiled into Python/C++ classes by `colcon build`
- Standard libraries: `std_msgs`, `sensor_msgs`, `geometry_msgs`

Go1 example: `/joint_states` → `sensor_msgs/msg/JointState` - arrays of name, position, velocity, effort (12 joints)

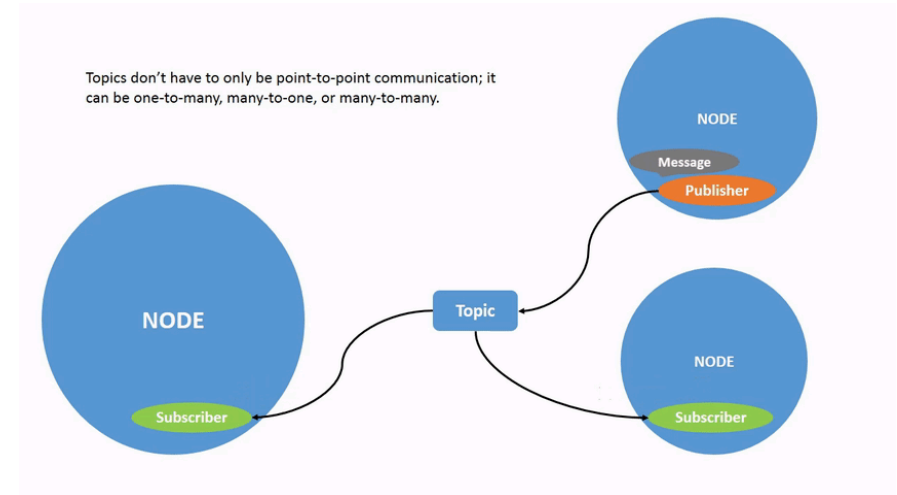
Kind	Extension	Used by
Message	<code>.msg</code>	Topics
Service	<code>.srv</code>	Services
Action	<code>.action</code>	Actions

Topic

Publish / Subscribe

- Named channel: publishers send, subscribers receive
- Many-to-many - no handshake, no coordination
- Non-blocking: messages are sent regardless of whether anyone is listening
- Best for: continuous sensor data, state streams

Go1 example: `/joint_states` - joint positions broadcast at 50 Hz

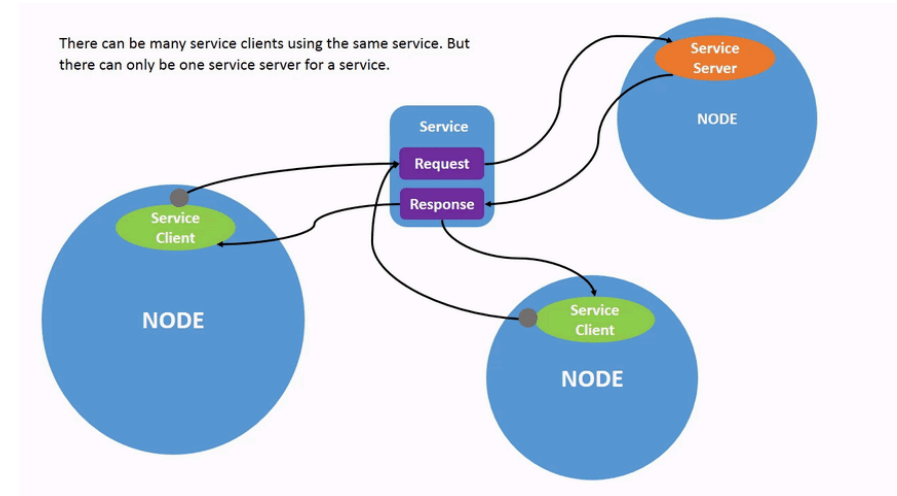


Service

Request / Response

- Client sends a request, waits for the server's reply
- Synchronous - the calling node blocks until done
- Single exchange - not suited for continuous data
- Best for: queries, configuration calls

Go1 example: "which poses are loaded?" → list returned immediately

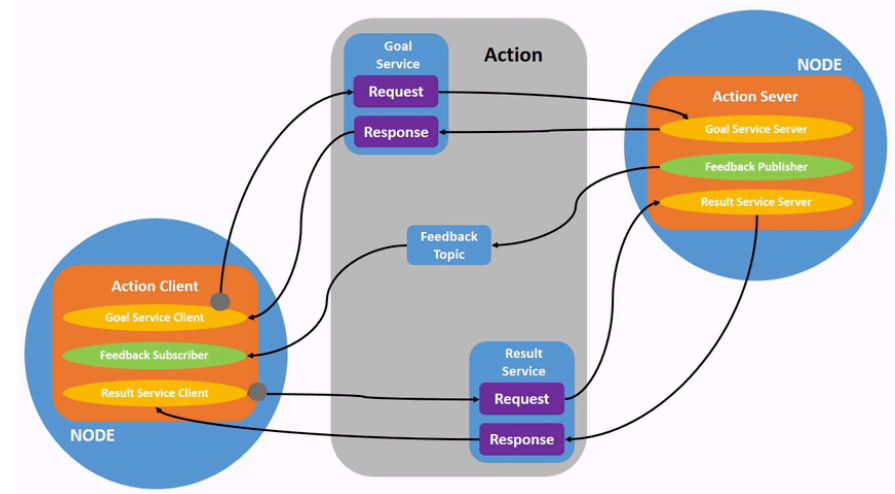


Action

Goal with feedback

- Client sends a **goal**; server accepts or rejects
- Server streams **feedback** while executing (progress, error, ...)
- Client can **cancel** mid-execution; server sends a final **result**
- Best for: long-running tasks where progress matters

Go1 example: "move to standing" - reports joint error at each step



Parameters

Runtime configuration

- Key-value store owned by each node
- Loaded from YAML at startup, tunable while running
- Other nodes can read or write via a parameter client
- Best for: gains, thresholds, any value requiring runtime adjustment

Go1 example: PD gains in `controller_pid.yaml` - edit the file, relaunch the controller

```
controller_manager:
  ros_parameters:
    update_rate: 500 # Hz

    # Joint state broadcaster to read joint position and velocity
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    joint_pid_controller:
      type: pid_controller/PidController

# PID controller for Go1 joints
joint_pid_controller:
  ros_parameters:
    dof_names: ["FL_hip_joint", "FL_thigh_joint", "FL_calf_joint",
                "FR_hip_joint", "FR_thigh_joint", "FR_calf_joint",
                "RL_hip_joint", "RL_thigh_joint", "RL_calf_joint",
                "RR_hip_joint", "RR_thigh_joint", "RR_calf_joint"]
    command_interface: "effort"
    reference_and_state_interfaces: ["position", "velocity"]
    use_external_measured_states: false

  gains:
    FL_hip_joint:
      p: 0.0
      i: 0.0
      d: 0.0
      antiwindup_strategy: "none"
    FL_thigh_joint:
      p: 0.0
      i: 0.0
      d: 0.0
      antiwindup_strategy: "none"
    FL_calf_joint:
      p: 0.0
      i: 0.0
      d: 0.0
      antiwindup_strategy: "none"
    FR_hip_joint:
      p: 0.0
```

Workspace Setup

Live demo - follow along in your terminal

Simulation

Gazebo and why we simulate at all

The Case for Simulation

Why not always use the real robot?

Real robot

Simulation

Expensive hardware, limited access

Runs on any laptop

Crashes damage the robot

Crashes are free

Hard to reproduce exact conditions

Fully deterministic and repeatable

Simulation enables fast iteration without hardware risk.

Simulation and Visualisation

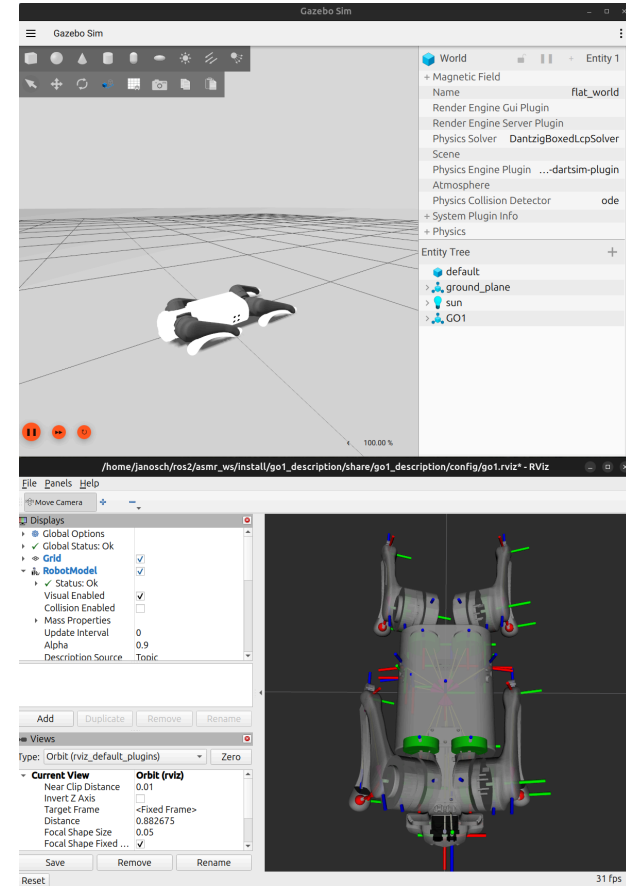
Two tools - different jobs

Gazebo - physics simulator

- The robot *exists* in a virtual world
- Gravity, inertia, contact forces are computed
- Controllers run and produce real joint torques
- Tune gains here before touching hardware

RViz - visualiser

- Displays ROS2 data: topics, transforms, sensor streams
- No physics - the robot does not fall, does not respond
- Used for debugging sensor data and transforms



Joint Control Architecture

`ros2_control` - the control layer

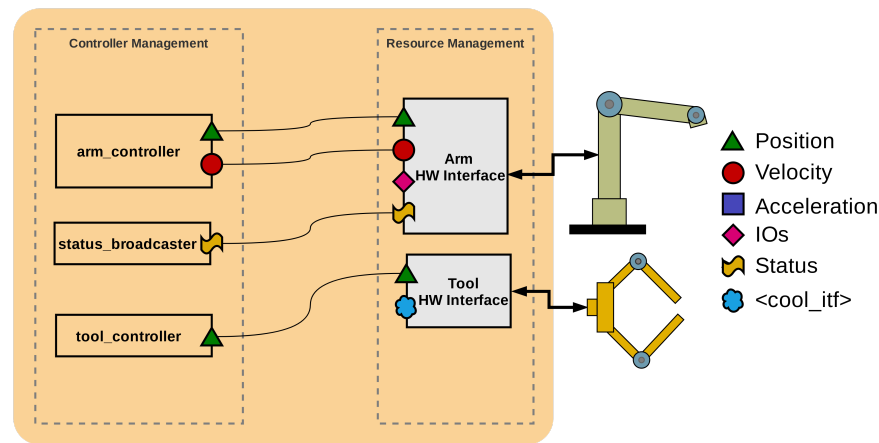
The Control Layer

The control layer

`ros2_control` sits between your ROS2 code and the robot's joints. It is the standard ROS2 framework - the same stack runs in Gazebo and on the real Go1.

Three layers, top to bottom:

- Your node publishes joint position targets to a topic
- Controller manager loads **broadcasters** (read joint states → topics) and **controllers** (receive targets → joint commands), running both at a fixed rate
- Resource manager owns the hardware interfaces and brokers access for controllers
- Hardware interface abstracts the physical joint



CC-BY: Denis Stogl, Bence Magyar (ros2_control)

Go1 Package Walkthrough

Live demo - package structure and controller configuration

Session Exercises

Notebook and First ROS2 Package

Task Overview

Jupyter Notebook

PD control — from theory to implementation

- Bang-bang $\rightarrow$ P $\rightarrow$ PD $\rightarrow$ PID progression
- 2D lane-following car simulation

ROS2 Exercise

Configure the Unitree Go1 and build your first ROS2 package

- **Go1**: tune PD gains and observe behaviour changes
- **Turtlesim**: draw a star — topics, services, and actions in practice

No session next week — public holiday. Two weeks to complete both exercises.